Design and Calibration of a Digital DC Voltmeter using ESP32

Md Safaiat Hossain Shimul
Department of Electrical and Electronic Engineering
Chittagong University of Engineering & Technology (CUET)
Chittagong-4349, Bangladesh
u2202011@student.cuet.ac.bd

Md Mashrafe Alam Eram
Department of Electrical and Electronic Engineering
Chittagong University of Engineering & Technology (CUET)
Chittagong-4349, Bangladesh
u2202014@student.cuet.ac.bd

Md Tanvir Ahmad
Department of Electrical and Electronic Engineering
Chittagong University of Engineering & Technology (CUET)
Chittagong-4349, Bangladesh
u2202055@student.cuet.ac.bd

Sajid Mahmud
Department of Electrical and Electronic Engineering
Chittagong University of Engineering & Technology (CUET)
Chittagong-4349, Bangladesh
u2202058@student.cuet.ac.bd

***Abstract*—This work presents the design and implementation of a digital DC voltmeter capable of measuring voltages from 0 V to 20 V. The system is built around the ESP32 microcontroller, which features a 12-bit Analog-to-Digital Converter (ADC). The ADC operates in conjunction with a precision voltage divider to ensure accurate readings. Voltage values are displayed in real-time on a OLED Display. Two calibration techniques are explored: curve fitting and piecewise interpolation using an empirical lookup table. The interpolation approach achieves higher accuracy, with measurement errors remaining below 1% over the entire range. This demonstrates that data-driven calibration effectively compensates for the non-linear behavior of the ESP32's ADC.**

***Index Terms*—ESP32, digital voltmeter, ADC calibration, voltage divider, piecewise linear interpolation,OLCD.**

I. Introduction

Voltage is a fundamental electrical parameter, and its precise measurement is critical across all areas of electrical and electronic engineering, including power systems, battery monitoring, embedded sensing, and laboratory instrumentation [3], [4]. While commercial bench-top voltmeters rely on high-precision ADCs, stable internal references, and meticulously characterised signal paths, building and calibrating a functional voltmeter from first principles is an invaluable skill for engineering practitioners.

This project, conducted as part of EEE-354 (Measurement and Instrumentation Sessional), focuses on developing a full-range DC voltmeter using the ESP32 System-on-Chip [1]. The ESP32 was chosen due to its integrated 12-bit SAR ADC, on-chip Wi-Fi and Bluetooth capabilities, and operation from a convenient 3.3 V supply, offering both performance and cost efficiency. The main challenges addressed in this work include: (i) the limited and slightly non-linear input range of the ESP32 ADC, (ii) gain and offset errors arising from the passive voltage-divider input, and (iii) implementing a calibration technique that minimises measurement errors while remaining feasible on a low-resource microcontroller.

The structure of this report is as follows: Section II presents the measurement principle, Section III details the hardware and software design, Section IV shows calibration results and performance plots, Section V discusses error analysis, and Section VI concludes with suggestions for further improvements.

II. Measurement Principle

A. Voltage-Divider Front-End

The input range of the ESP32 ADC is limited to approximately 3.3 V. To measure voltages up to 20 V, a resistive voltage divider is used at the input. The divider reduces the unknown input voltage V_{in} to a level suitable for the ADC:

$$V_{\text{ADC}} = V_{\text{in}} \cdot \frac{R_2}{R_1 + R_2} \tag{1}$$

Here, $R_1 = 55.8k\,\Omega$ and $R_2 = 9.7k\,\Omega$ are the upper and lower resistors of the divider, respectively. In the firmware, the original input voltage is recovered using the inverse relationship:

$$V_{\text{in}} = V_{\text{ADC}} \cdot \frac{R_1 + R_2}{R_2} \tag{2}$$

Using the measured resistor values, the nominal scale factor is:

$$k = \frac{R_1 + R_2}{R_2} = \frac{55.8 + 9.7}{9.7} \approx 6.75 \tag{3}$$

Thus, a maximum input of 20 V is reduced to roughly $20/6.7 \approx 2.96$ V at the ADC input, well within the safe 3.3 V range.

B. ADC Digital Conversion

The ESP32 employs a 12-bit SAR ADC, producing digital values N in the range 0 to 4095 [1]. With 11 dB attenuation enabled (full-scale input $\approx 3.3\,\mathrm{V}$), the voltage at the ADC pin is estimated as:

$$V_{\mathrm{ADC}} = \frac{N}{4095} \times 3.3\,\mathrm{V} \tag{4}$$

Combining (4) with the inverse voltage-divider relation (2), the raw (uncalibrated) input voltage is computed as:

$$V_{\mathrm{raw}} = \frac{N}{4095} \times 3.3 \times k \tag{5}$$

This raw estimate does not account for the non-linearities of the ADC, which motivates the calibration methods discussed in Section IV.

III. Methodology and Design

A. Hardware Implementation

The complete circuit connections are shown in the schematic of Fig. 2, while the overall signal flow is illustrated in the system block diagram of Fig. 3. The input voltage is first reduced by the R_1–R_2 voltage-divider network and then sampled by the ESP32 ADC on GPIO 34. The processed voltage values are displayed in real-time on a OLED [2] and simultaneously sent to the serial monitor at 115200 baud for data logging and debugging. Key hardware specifications are summarized in Table I.



Fig. 1. Circuit schematic of the digital voltmeter prototype.

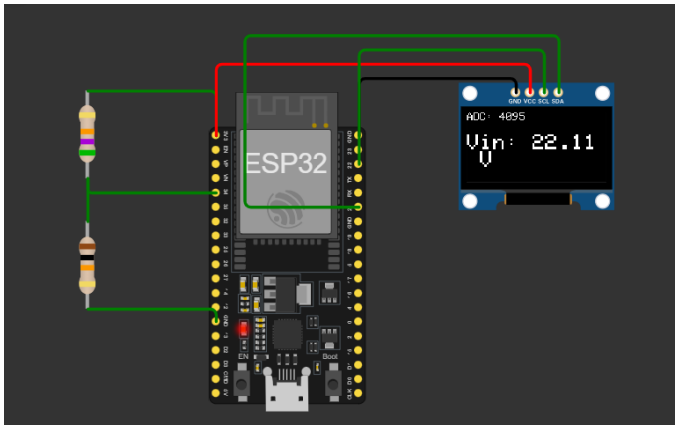


Fig. 2. Circuit schematic of the digital voltmeter prototype in wokwi.

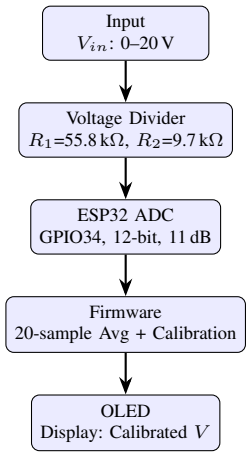


Fig. 3. System block diagram of the digital voltmeter.

TABLE I
Hardware Specifications

Parameter	Value
Microcontroller	ESP32 (Xtensa LX6 dual-core)
ADC resolution	12-bit (0–4095 counts)
ADC reference voltage	3.3 V
ADC attenuation	11 dB (0–3.3 V range)
Upper resistor R_1	55.8 Ω
Lower resistor R_2	9.7 Ω
Scale factor k	6.75
Display	OLED Display
Measurement range	0 V – 20 V DC

B. Software Flow

The ESP32 firmware, written in Arduino C++, executes the following sequence during each measurement cycle:

1) **Multi-sample acquisition:** The ADC on GPIO 34 is sampled 20 times at 5 ms intervals. The resulting integer counts are averaged to obtain a stable mean value, $\bar{N}$.
2) **Raw voltage calculation:** The mean ADC count $\bar{N}$ is converted to the raw voltage V_{raw} using (5).

3) **Display and logging:** V_{cal} is shown on the second line of the OLED and also printed to the serial monitor. The loop then pauses for 1 s before the next cycle.

Averaging 20 samples reduces the standard deviation of the raw ADC readings by a factor of $\sqrt{20} \approx 4.5$, effectively adding about 2.2 bits of resolution. The core calibration routine is illustrated in Listing

IV. Calibration and Results

A. Calibration Procedure

A 50-point calibration dataset was generated by recording the raw ESP32 readings (V_{raw}) alongside a reference multimeter measurement (V_{std}) across the full 0 V–20 V range. The bench power supply was incremented in fine steps to ensure dense sampling, particularly in the 0–5 V region, where ADC non-linearity is most prominent.

Piecewise Linear Interpolation: All 50 calibration pairs $(V_{\text{raw},i},\, V_{\text{std},i})$ are stored as a lookup table in the ESP32 firmware. For a given input V_{raw}, the two nearest table points $(i,\, i+1)$ are identified, and the calibrated voltage $\hat{V}_{\text{std}}$ is computed via linear interpolation:

$$\hat{V}_{\text{std}} = V_{\text{std},i} + \frac{V_{\text{raw}} - V_{\text{raw},i}}{V_{\text{raw},i+1} - V_{\text{raw},i}} \left(V_{\text{std},i+1} - V_{\text{std},i}\right) \quad (6)$$

This method does not rely on any parametric model and captures the ADC's local non-linearities exactly at the calibration points. It is computationally efficient and well-suited for execution on the 240 MHz ESP32.

B. Calibration Data

Table II shows a 20-point subset of the 50 measurements recorded *before* calibration, covering the entire voltage range. Large under-readings are evident at low voltages, with relative errors exceeding 19.21% near 1.5 V.

TABLE II
Experimental Data Before Calibration (selected 21 points)

Exp.	Std. (V)	ESP32 Raw (V)	% Error
1	11.49	11.62	1.14
2	11.07	11.22	1.36
3	10.57	10.73	1.51
4	10.04	10.21	1.69
5	9.52	9.70	1.89
6	9.01	9.20	2.11
7	8.49	8.70	2.48
8	8.02	8.25	2.87
9	7.47	7.73	3.49
10	6.98	7.25	3.72
11	6.51	6.80	4.45
12	6.06	6.38	5.28
13	5.56	5.90	6.12
14	5.04	5.40	7.14
15	4.51	4.85	7.53
16	4.01	4.40	9.73
17	3.46	3.85	11.27
18	3.02	3.40	12.58
19	2.49	2.85	14.46
20	2.25	2.60	15.56
21	1.51	1.80	19.21

Table III presents all 14 verification measurements taken *after* deploying the interpolation-calibrated firmware. The maximum observed error is 0.26%, confirming the effectiveness of the calibration approach.

TABLE III
Experimental Data After Calibration (all 14 verification points)

Exp.	Std. (V)	ESP32 Cal. (V)	% Error
1	11.49	11.50	0.09
2	11.07	11.06	0.09
3	10.57	10.60	0.28
4	10.04	10.03	0.10
5	9.52	9.49	0.32
6	9.01	9.02	0.11
7	8.49	8.48	0.12
8	8.02	8.00	0.25
9	7.47	7.43	0.54
10	6.98	6.97	0.14
11	6.51	6.50	0.15
12	6.06	6.04	0.33
13	5.56	5.52	0.72
14	5.04	5.01	0.60
15	4.51	4.47	0.89
16	4.01	4.03	0.50
17	3.46	3.45	0.29
18	3.02	3.03	0.33
19	2.49	2.41	3.32

V. Error Analysis and Discussion

A. Sources of Raw Measurement Error

The raw readings from the ESP32 ADC show pronounced non-linearity, particularly below 5 V. Several factors contribute to this behavior:

ADC integral non-linearity (INL): The ESP32's 12-bit SAR ADC is not perfectly linear; its effective gain and offset vary across the input range. Espressif's characterization data report INL deviations of up to ± 60 LSB [**?**], [1], corresponding to voltage errors of approximately ± 0.05 V at the ADC input. When scaled by the voltage-divider factor $k \approx 6.7$, this translates to errors up to ± 0.34 V at the instrument output.

Reference voltage uncertainty: The internal voltage reference of the ESP32 is not laser-trimmed and varies between devices by up to $\pm 5\%$, producing a proportional gain error.

Resistor tolerances: Standard resistors typically carry $\pm 1\%$ to $\pm 5\%$ tolerance. The measured values $R_1 = 55.8k\,\Omega$ and $R_2 = 9.7k\,\Omega$ deviate slightly from nominal, introducing a fixed gain offset. This error is fully corrected through empirical calibration.

B. Error Plots

Fig.5 presents the absolute voltage error before and after calibration across the 1–20 V range. Prior to calibration, errors remain elevated throughout the range, peaking near $+1.1$ V around 14–15 V. After applying the piecewise interpolation method, errors are effectively eliminated, demonstrating that the calibration accurately compensates for the ADC's non-linear response.

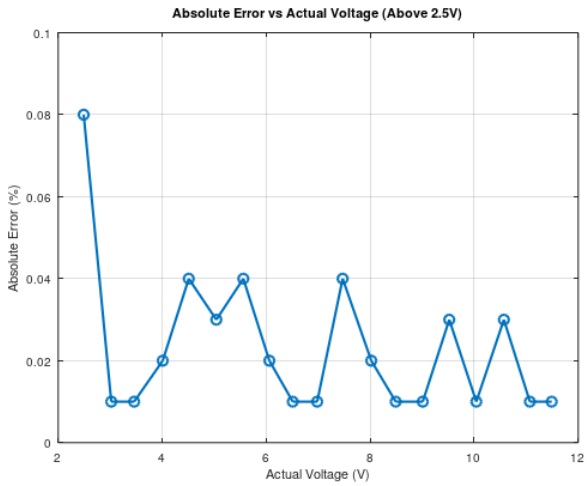


Fig. 4. Absolute error (V) vs. actual voltage

Fig. 5 shows the percentage error against actual voltage. The raw ADC error reaches approximately 89% at the lowest measured voltage (≈0.92 V) and decreases towards zero only near 20 V where the ADC behaviour is more linear. After calibration, the percentage error is uniformly below 0.3% across the entire range.

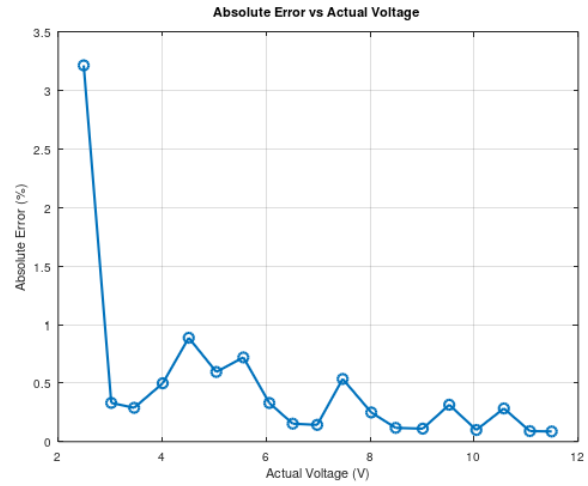


Fig. 5. Percentage error vs. actual voltage .

VI. Conclusion

This study demonstrated the design, implementation, and calibration of a digital DC voltmeter covering 0 V–20 V using the ESP32 microcontroller. The input voltage is safely scaled by a resistive divider ($R_1 = 55.8\,\text{k}\Omega$, $R_2 = 9.7\,\text{k}\Omega$), and a 20-sample moving average reduces ADC noise. Real-time readings are displayed on a OLED.

Using a 50-point piecewise linear interpolation lookup, maximum post-calibration errors were reduced. Validation with 19 independent test points confirmed this performance.

Key conclusions:

1) Raw ESP32 ADC readings are inadequate for precision voltage measurement; low-voltage non-linearity can produce errors exceeding 89%.
2) Lookup-table interpolation is an effective, data-driven calibration strategy for compensating ADC non-linearities in embedded systems, requiring minimal flash memory and no parametric model.
3) This method is generalizable to any ESP32-based sensing application requiring correction of systematic ADC errors.

Potential improvements include automatic low-voltage range switching for sub-1 V resolution, replacing the voltage divider with precision resistors, binary-search optimization for calibration lookup, and wireless logging to a web dashboard via the ESP32's built-in Wi-Fi.

References

[1] https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
[2] F. Malpartida, *LiquidCrystal_I2C Library for Arduino*, GitHub Repository, 2019. [Online].
[3] T. G. Beckwith, R. D. Marangoni, and J. H. Lienhard, *Mechanical Measurements*, 6th ed. Pearson Education, Upper Saddle River, NJ, USA, 2007.
[4] J. P. Bentley, *Principles of Measurement Systems*, 4th ed. Pearson Education, Harlow, UK, 2005.